

E-Payment System

Code Documentation & Technical Reference

Based on: E-Payment System to Reduce Use of Paper Money for Daily Transactions

ICECTE 2022, Rajshahi-6204, Bangladesh

Extended with TLS 1.3 Integration for Production-Grade Security

Version: MVP 1.0 | May 2026

Stack: Next.js 14 + TypeScript + Flask + PostgreSQL + Redis + Nginx + Docker

Table of Contents

- 1. System Overview**
- 2. Project Structure**
- 3. Technology Stack**
- 4. Database Schema**
 - 4.1 Users & Accounts
 - 4.2 Device Credentials & Timestamp Keys
 - 4.3 Transactions & Audit Log
 - 4.4 TLS Certificates & Notifications
- 5. Cryptographic Architecture**
 - 5.1 Key Hierarchy (K1, K2, Session Secret, T)
 - 5.2 Transaction Encryption Pipeline
 - 5.3 Server-Side Verification Pipeline
- 6. API Specification**
 - 6.1 Authentication Endpoints
 - 6.2 Transaction Endpoints
 - 6.3 Account & Device Endpoints
 - 6.4 Notification & Simulation Endpoints
- 7. Frontend Architecture (Next.js 14)**
 - 7.1 Page Structure & Routing
 - 7.2 Component Library
 - 7.3 BFF Layer (API Route Handlers)
 - 7.4 Crypto Library (Client-Side)
- 8. Security Architecture**
 - 8.1 Four-Layer Security Model
 - 8.2 Session & Authentication Security
 - 8.3 Replay Prevention
 - 8.4 Security Hardening Applied
- 9. Deployment Architecture (Docker)**
- 10. Paper Compliance Verification**

1. System Overview

The E-Payment System is a web-based digital payment platform designed to replace physical paper money for everyday petty cash transactions. It implements the cryptographic framework proposed in the ICECTE 2022 paper with a two-layer security architecture combining TLS 1.3 at the transport layer and AES-256-GCM with HMAC-SHA256 at the application layer.

The system enables users to register (via bank officer), authenticate, check balances, send encrypted payments to other users, view transaction history, and manage their profiles. Every payment is cryptographically signed with a device-bound HMAC key (K1), encrypted with an AES key derived from the user password (K2) and a session secret (Bp substitute), and protected against replay attacks via a rolling timestamp key (T).

Core Principles: Confidentiality, Integrity, Authentication, Non-Repudiation.

Key Metrics

Metric	Value
Database Tables	8 (users, accounts, device_credentials, timestamp_keys, transactions, audit_log, tls_certificates, notifications)

E-Payment System -- Code Documentation

API Endpoints	16 (auth:4, transaction:4, account:2, device:2, health:1, notification:3, simulate:2)
Frontend Pages	7 (landing, login, dashboard, send, history, simulator, profile)
Reusable Components	10 (DashboardShell, BalanceCard, StatCard, QuickActionCard, TransactionRow, EmptyState, LoadingSkeleton, NotificationDropdown, Backg
Security Layers	4 (Transport, Confidentiality, Integrity, Auth)
Cryptographic Keys	4 (K1, K2, Session Secret, T)

2. Project Structure

```
epayment-mvp/  
|-- backend/                                # Flask API server  
|   |-- app/  
|       |-- __init__.py                    # App factory, blueprint registration  
|       |-- extensions.py                 # SQLAlchemy, Redis, JWT blacklist  
|       |-- config.py                     # Environment configuration  
|       |-- models/                       # SQLAlchemy ORM models (7 tables)  
|       |-- routes/                       # Flask blueprint route handlers  
|       |-- services/                     # Business logic (crypto, auth, transaction, audit)  
|       |-- middleware/                   # JWT auth, rate limiting  
|       |-- tasks/                        # Celery async tasks (daily reset)  
|   |-- seeds/seed.py                     # Idempotent test user seeder  
|   |-- migrations/                       # Alembic migration scripts  
|   |-- wsgi.py                           # Gunicorn/Flask entry point  
|   |-- requirements.txt                  # Python dependencies  
|  
|-- frontend/                              # Next.js 14 application  
|   |-- app/  
|       |-- page.tsx                      # Landing page (/)  
|       |-- login/page.tsx                # Login page  
|       |-- dashboard/                    # Protected dashboard pages  
|       |-- api/                           # BFF route handlers  
|       |-- globals.css                   # Tailwind CSS + theme  
|   |-- components/                       # Reusable React components  
|   |-- lib/                              # Crypto, session, CSRF utilities  
|   |-- types/index.ts                    # TypeScript interfaces  
|   |-- middleware.ts                     # Next.js edge middleware  
|  
|-- nginx/nginx.conf                      # Nginx reverse proxy + TLS config  
|-- docker-compose.yml                    # Multi-service orchestration  
|-- schema.dbml                           # DBML database schema  
|-- supabase_schema.sql                   # PostgreSQL SQL schema
```

3. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router) + TypeScript	SSR, BFF pattern, type-safe crypto
Styling	Tailwind CSS v4 + framer-motion	Rapid UI, animations, glassmorphism
Auth (Frontend)	iron-session v8	Encrypted httpOnly cookie sessions
Crypto (Frontend)	Node.js crypto module	AES-256-GCM, HMAC-SHA256, HKDF
Backend API	Python 3.12 + Flask 3.1	RESTful API, middleware, blueprints
ORM	SQLAlchemy 2.0 + Flask-Migrate	Type-safe queries, migrations
Crypto (Backend)	cryptography + PyJWT + bcrypt	AES-GCM, Fernet, JWT, password hash
Database	PostgreSQL 16 (Supabase managed)	ACID transactions, NUMERIC(15,2), FOR UPDATE
Cache/Queue	Redis 7 + Celery 5	JWT blacklist, rate limiter, async tasks

Reverse Proxy	Nginx (Alpine)	TLS 1.3 termination, rate limiting, HSTS, CSP
Container	Docker + Docker Compose	Multi-service orchestration, dev/prod parity

4. Database Schema

4.1 Users & Accounts

The users table stores core identity. Raw NID/BRC numbers are never stored -- only their SHA-256 hash. The accounts table is separated from users so database roles can restrict financial data access independently from identity data.

Table	Columns	Key Constraints
users	user_id(UUID), username, full_name, nid_hash, password	PK: user_id, UNIQUE: username, nid_hash
accounts	account_id(UUID), user_id(FK), balance, daily_limit, daily_tx_count	PK: account_id, FK: user_id->users, CHECK: balance>=0

4.2 Device Credentials & Timestamp Keys

Cryptographic credentials tied to a specific registered device/browser session. The timestamp_keys table stores the anti-replay state -- T current value and version counter.

Table	Key Columns	Security Notes
device_credentials	device_id, user_id, browser_fingerprint(hashed), k1_hash, session_secret	K1 only stored as hash. K2 + session_secret Fernet-encrypted.
timestamp_keys	ts_key_id, user_id, current_t, t_version, last_updated_at	T updated via HMAC chain after each tx. Version prevents replays.

4.3 Transactions & Audit Log

The transactions table is an immutable financial ledger -- records are never updated or deleted. The audit_log table is append-only, recording every security-relevant event.

Table	Purpose	Key Columns
transactions	Immutable financial ledger	transaction_id, sender_id, receiver_id, amount(NUMERIC), status, hmac_verified, t_version_used, initiated_at
audit_log	Security event logging	log_id, user_id, event_type, event_detail(JSONB), ip_address, occurred_at

4.4 TLS Certificates & Notifications

TLS certificates table tracks issued mTLS client certificates (future use). The notifications table stores user notifications triggered by incoming transfers and system events.

5. Cryptographic Architecture

5.1 Key Hierarchy

The system uses four cryptographic keys, each with a distinct purpose:

Key	Derivation	Storage	Purpose
K1	HMAC-SHA256(activation_code, SHA256(NID)	SHA256(K1) in device_credentials; raw K1 in	HMAC identity key -- proves message originated from registered
K2	User-chosen password	bcrypt hash for auth; Fernet-encrypted raw in	Authentication credential + AES key material
Session Secret	os.urandom(32).hex() (256-bit)	SHA256 hash in device_credentials; Fernet-e	Device-bound session secret -- Bp substitute for web MVP
T (Timestamp)	HMAC chain: T_n = HMAC(server_secret, T_{n-1}	timestamp_keys.current_t	Anti-replay -- changes after every transaction

5.2 Transaction Encryption Pipeline (Client/BFF Side)

The encryption pipeline runs in the Next.js BFF layer (Node.js runtime) before forwarding to Flask:

1. K1 = deriveK1(activation_code_hash, nid_hash, browser_fingerprint)
2. M = JSON{sender_username, receiver_username, amount, currency, timestamp, nonce}

```

3. F1 = HMAC-SHA256(K1, serialize(M)) ...integrity signature
4. AES_Key = HKDF-SHA256(K2 || session_secret || t_current, nonce, info='epayment-transaction-v1')
5. AAD = sender_username + t_version ...context binding
6. (IV, ciphertext, auth_tag) = AES-256-GCM.encrypt(AES_Key, IV(96-bit), M||F1, AAD)
7. encrypted_payload = Base64(IV || ciphertext || auth_tag)
8. POST /api/v1/transaction/initiate { encrypted_payload, declared_t_version, device_id }

```

5.3 Server-Side Verification Pipeline (Flask)

```

1. JWT verify: signature, expiry, jti not blacklisted
2. Load sender, device (check active), account (FOR UPDATE lock), timestamp_key (FOR UPDATE lock)
3. REPLAY CHECK: declared_t_version == ts_key.t_version ?
4. Derive K1 from stored hash components: K1 = HMAC(act_hash, nid_hash, browser_fp_hash)
5. Decrypt K2_raw + session_secret_raw from Fernet-encrypted DB fields
6. Derive AES_Key = HKDF-SHA256(K2_raw || session_secret_raw || t_current, nonce, info)
7. AES-GCM.decrypt(encrypted_payload, AAD) -> verify GCM auth tag (INTEGRITY CHECK 1)
8. Parse plaintext: M_str | F1_received
9. HMAC verify: F2 = HMAC-SHA256(K1, M_str), compare_digest(F1_received, F2) (CHECK 2)
10. Business: receiver exists? sender.balance >= amount? daily_limit not exceeded?
11. ATOMIC POSTGRES: UPDATE sender.balance-=amount, receiver.balance+=amount, INSERT tx, UPDATE timestamp_key, INSERT notification, ...

```

6. API Specification

All endpoints served over HTTPS (TLS 1.3 via Nginx). Authenticated endpoints require Authorization: Bearer <jwt>. The Next.js BFF layer proxies browser requests to Flask -- the browser never calls Flask directly.

6.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	Officer API Key	Register new user (bank officer)
POST	/api/v1/auth/login	None (rate-limited)	Authenticate user, return JWT + crypto materials
POST	/api/v1/auth/logout	JWT	Blacklist JWT jti in Redis
POST	/api/v1/auth/refresh	JWT	Issue new JWT before expiry

6.2 Transaction Endpoints

Method	Endpoint	Description
POST	/api/v1/transaction/initiate	Core endpoint: receive AES-GCM encrypted payload, verify HMAC, process atomically
POST	/api/v1/transaction/initiate-bff	BFF-friendly: accepts raw data, encrypts internally (bypasses client crypto)
GET	/api/v1/transaction/history	Paginated transaction history (?page=&limit=)
GET	/api/v1/transaction/:id	Single transaction detail (sender/receiver only)

6.3 Account, Device, Notification & Simulation Endpoints

Method	Endpoint	Description
GET	/api/v1/account/balance	Return balance, daily limit, daily used, daily remaining
POST	/api/v1/account/suspend	Self-suspend: set account_status='suspended', blacklist tokens
GET	/api/v1/device/status	Device active status, registration date, last used
GET	/api/v1/health	Health check: DB + Redis connectivity status
GET	/api/v1/notification/list	List notifications (?limit=&offset=)
GET	/api/v1/notification/unread-count	Count of unread notifications
POST	/api/v1/notification/read	Mark one or all notifications as read

POST	/api/v1/simulate/encrypt	Simulate encryption side, return all intermediate crypto values
POST	/api/v1/simulate/decrypt	Simulate decryption side, return verification results

7. Frontend Architecture (Next.js 14)

7.1 Page Structure & Routing

Route	File	Auth	Description
/	app/page.tsx	Public	Landing page with hero, features, CTA
/login	app/login/page.tsx	Public	Login form with demo credentials
/dashboard	app/dashboard/page.tsx	JWT required	Balance overview, quick actions, recent txns
/dashboard/send	app/dashboard/send/page.tsx	JWT required	Send money form with crypto step indicator
/dashboard/history	app/dashboard/history/page.tsx	JWT required	Paginated transaction list with filters
/dashboard/simulate	app/dashboard/simulate/page.tsx	JWT required	Crypto pipeline visualizer
/dashboard/profile	app/dashboard/profile/page.tsx	JWT required	User profile, device status, limits

7.2 Component Library

The frontend uses 10 reusable components following the glassmorphism design system:

Component	File	Purpose
DashboardShell	components/DashboardShell.tsx	Sidebar + topbar layout wrapper
BalanceCard	components/BalanceCard.tsx	Blue gradient hero with daily limit progress
StatCard	components/StatCard.tsx	Icon + label + value stat display
QuickActionCard	components/QuickActionCard.tsx	Animated link card with hover physics
TransactionRow	components/TransactionRow.tsx	Single transaction row with direction + status
EmptyState	components/EmptyState.tsx	Empty state with icon + message
LoadingSkeleton	components/LoadingSkeleton.tsx	Spinner / card / table skeleton variants
NotificationDropdown	components/NotificationDropdown.tsx	Bell dropdown with polling
BackgroundParticles	components/BackgroundParticles.tsx	Animated background effects

7.3 BFF Layer (API Route Handlers)

The Backend-for-Frontend (BFF) pattern isolates the browser from Flask. Next.js Route Handlers proxy requests, handle session management, perform client-side crypto, and manage CSRF tokens.

BFF Route	Proxies To	Extra Logic
POST /api/auth/login	Flask /login	Sets CSRF cookie, stores crypto materials in iron-session
POST /api/transaction	Flask /initiate	AES-256-GCM encrypts payload before forwarding
GET /api/account/balance	Flask /balance	Bearer token forwarding
POST /api/auth/logout	Flask /logout	Destroys iron-session after Flask blacklist

7.4 Crypto Library (Client-Side)

The frontend crypto library (frontend/lib/crypto.ts) mirrors the backend crypto_service.py exactly. Functions:

```

deriveK1(codeHash, nidHash, browserFp) -> Buffer
computeHmac(k1Hex, message)             -> string (base64)
deriveAesKey(k2, secret, t, nonce)       -> Buffer
encryptPayload({sender, receiver, amount, k2, secret, t, ...}) -> {encryptedPayload, nonce}
hashSha256(input)                       -> string (hex)

```

8. Security Architecture

8.1 Four-Layer Security Model

Layer	Mechanism	What It Protects Against
Layer 4: Auth	JWT (HS256, 15min) + bcrypt + CSRF + officer key	Unauthorized access, brute force, CSRF
Layer 3: Integrity	HMAC-SHA256 (F1) + GCM auth tag + t_version	Message tampering, replay attacks
Layer 2: Confidentiality	AES-256-GCM + HKDF + AAD binding	Message reading, ciphertext manipulation
Layer 1: Transport	TLS 1.3 (Nginx) + HSTS + CSP	Network eavesdropping, MITM, downgrade

8.2 Session & Authentication Security

Sessions use iron-session (encrypted, httpOnly, SameSite=Strict cookies). JWTs are HS256-signed, expire in 15 minutes, and use jti-based blacklisting (Redis with in-memory fallback). CSRF protection uses a double-submit pattern: a readable csrf-token cookie validated against the X-CSRF-Token header on all state-changing requests.

8.3 Replay Prevention

The timestamp key T is an HMAC chain. After each successful transaction, T is updated via `derive_next_t()` using the server-side HMAC secret. The `t_version` counter must match the declared version exactly -- any mismatch is logged as `replay_detected` in the audit log. The `TimestampKey` row is locked with `SELECT FOR UPDATE` to prevent race conditions between concurrent requests (two workers cannot both pass the version check).

8.4 Security Hardening Applied

Finding	Severity	Fix Applied
SHA-256 as password hash	CRITICAL	Replaced with bcrypt (salted, adaptive cost)
Pass-the-hash (client hashing)	CRITICAL	Removed client hashing; raw password over TLS, bcrypt server-side
No TLS on Nginx	CRITICAL	Added TLS 1.3 + HSTS + CSP + self-signed dev cert
Spoofable browser fingerprint	CRITICAL	Server-side fingerprint from User-Agent + Accept-Language
Weak session secret	CRITICAL	Generated 256-bit cryptographically random secret
Double-spend race condition	HIGH	Added FOR UPDATE lock on <code>TimestampKey</code> row
Hardcoded nonce in dead code	HIGH	Deleted unused <code>verify_payload()</code> function
Weak config fallback defaults	HIGH	Removed all defaults; crashes on missing env vars
No CSRF token	HIGH	Added csrf-token cookie + X-CSRF-Token header validation
In-memory rate limiter	HIGH	Migrated to Redis-backed sorted sets per endpoint

9. Deployment Architecture (Docker)

The system runs as four Docker services orchestrated by `docker-compose.yml`. Only Nginx is publicly exposed. Backend, frontend, and Redis communicate on an internal bridge network.

```
services:
  backend      (Flask+Gunicorn, port 5000 internal)
    env: DATABASE_URL, JWT_SECRET, SERVER_HMAC_SECRET, REDIS_URL
    depends_on: redis (healthy)

  frontend     (Next.js standalone, port 3000 internal)
    env: API_BASE_URL=http://backend:5000, SESSION_SECRET
    depends_on: backend (healthy)

  redis        (Redis 7 Alpine, port 6379 internal)
    persistence: AOF, volume-mounted
```

```

nginx      (Nginx Alpine, ports 80+443 exposed)
  TLS: self-signed (dev), Let's Encrypt (production)
  rate_limit: auth=5r/s, api=30r/s
  headers: HSTS, CSP, XFO, XCTO, RP, Permissions-Policy

```

How to run:

```

# 1. Configure environment
cp backend/.env.example backend/.env    # Fill Supabase DATABASE_URL + secrets
cp frontend/.env.example frontend/.env.local

# 2. Start all services
docker compose up --build -d

# 3. Health check
curl http://localhost/api/v1/health

# 4. Seed demo users
docker exec epayment-backend python seeds/seed.py

```

10. Paper Compliance Verification

Each component from the ICECTE 2022 framework is verified against the implementation:

#	Paper Requirement	Implementation	Status
1	K1 = HMAC(act_code, NID MAC BP)	HMAC(act_code, NID browser_fp)	DONE (web adaptation)
2	K2 = user password	bcrypt auth + Fernet-encrypted for AES	DONE
3	BP = biometric	Random 32B session_secret	DONE (web adaptation)
4	F1 = HMAC(K1, M)	HMAC-SHA256 on both sides	DONE
5	AES encrypt M F1 (K2+BP+T)	AES-256-GCM (K2+secret+T)	DONE
6	Transmit insecure channel	TLS 1.3 + inner AES-GCM	DONE
7	Server decrypts M F1	AES-GCM decrypt + GCM verify	DONE
8	Server verifies HMAC	HMAC compare_digest	DONE
9	Check balance, debit/credit	SELECT FOR UPDATE, atomic commit	DONE
10	Update T after each tx	HMAC-chain T_new, version++	DONE

100% of the paper's cryptographic framework is implemented. Two adaptations (MAC->browser_fp, BP->session_secret) are documented web MVP trade-offs. The system also adds 10+ security enhancements beyond the paper specification including TLS 1.3, bcrypt password hashing, CSRF protection, Redis-backed rate limiting, audit logging, notification system, and concurrent safety via row-level locking.